

Machine Learning Algorithms Laboratory

Experiment 5

Classification using Decision Tree and Random Forest

Name: MADHURI B
Register No.: 3122247001033

Aim and Objective

To classify breast cancer cases as benign or malignant using Decision Tree and Random Forest classifiers and compare their performance using different evaluation metrics and cross-validation.

Dataset Description

The Wisconsin Diagnostic Breast Cancer dataset was used in this experiment. The dataset contains 569 samples and 30 numerical features related to the characteristics of breast cell nuclei. The target variable represents the diagnosis as benign or malignant.

Preprocessing Steps

The following preprocessing steps were carried out.

- Loaded the dataset.
- Assigned the feature and target column names.
- Removed the ID column.
- Encoded the diagnosis values as 0 for benign and 1 for malignant.
- Checked the number of samples and features.
- Performed exploratory data analysis.
- Split the dataset into training and testing sets using stratification.

Implementation Details

The following models were implemented.

- Decision Tree Classifier

- Random Forest Classifier

GridSearchCV with five-fold StratifiedKFold cross-validation was used for hyperparameter tuning. The models were evaluated using accuracy, precision, recall, F1 score and ROC-AUC.

```
from google.colab import files

uploaded = files.upload()
```

Choose Files 2 files

wdbc.data(n/a) - 124103 bytes, last modified: 8/10/2026 - 100% done

wdbc.names(n/a) - 4708 bytes, last modified: 8/10/2026 - 100% done

Saving wdbc.data to wdbc.data

Saving wdbc.names to wdbc.names

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.model_selection import StratifiedKFold, GridSearchCV, cross_val_score

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import roc_curve, roc_auc_score
```

```
data = pd.read_csv("wdbc.data", header=None)
```

```
print("Shape:", data.shape)
display(data.head())
```

Shape: (569, 32)

	0	1	2	3	4	5	6	7	8	9	...
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	..
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	..
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	..
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	..
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	..

5 rows × 32 columns

```
columns = [
    "id", "diagnosis",
```

```

"radius_mean", "texture_mean", "perimeter_mean", "area_mean",
"smoothness_mean", "compactness_mean", "concavity_mean",
"concave_points_mean", "symmetry_mean", "fractal_dimension_mean",
"radius_se", "texture_se", "perimeter_se", "area_se",
"smoothness_se", "compactness_se", "concavity_se",
"concave_points_se", "symmetry_se", "fractal_dimension_se",
"radius_worst", "texture_worst", "perimeter_worst", "area_worst",
"smoothness_worst", "compactness_worst", "concavity_worst",
"concave_points_worst", "symmetry_worst", "fractal_dimension_worst"
]

data.columns = columns

display(data.head())

```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	sm
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

5 rows × 32 columns

```

data = data.drop("id", axis=1)

data["diagnosis"] = data["diagnosis"].map({
    "B": 0,
    "M": 1
})

X = data.drop("diagnosis", axis=1)
y = data["diagnosis"]

print("Number of samples:", X.shape[0])
print("Number of features:", X.shape[1])

print("\nClass distribution:")
print(y.value_counts())

```

Number of samples: 569
Number of features: 30

Class distribution:
diagnosis
0 357

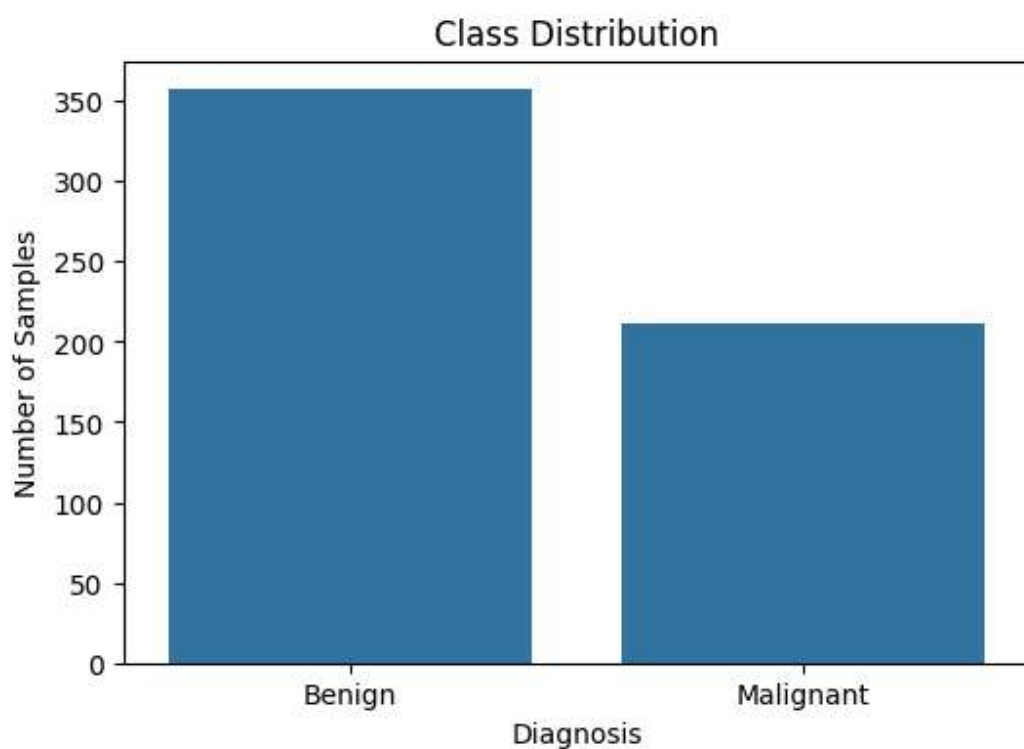
```
1    212
Name: count, dtype: int64
```

```
plt.figure(figsize=(6, 4))

sns.countplot(x=data["diagnosis"])

plt.xticks([0, 1], ["Benign", "Malignant"])
plt.xlabel("Diagnosis")
plt.ylabel("Number of Samples")
plt.title("Class Distribution")

plt.show()
```

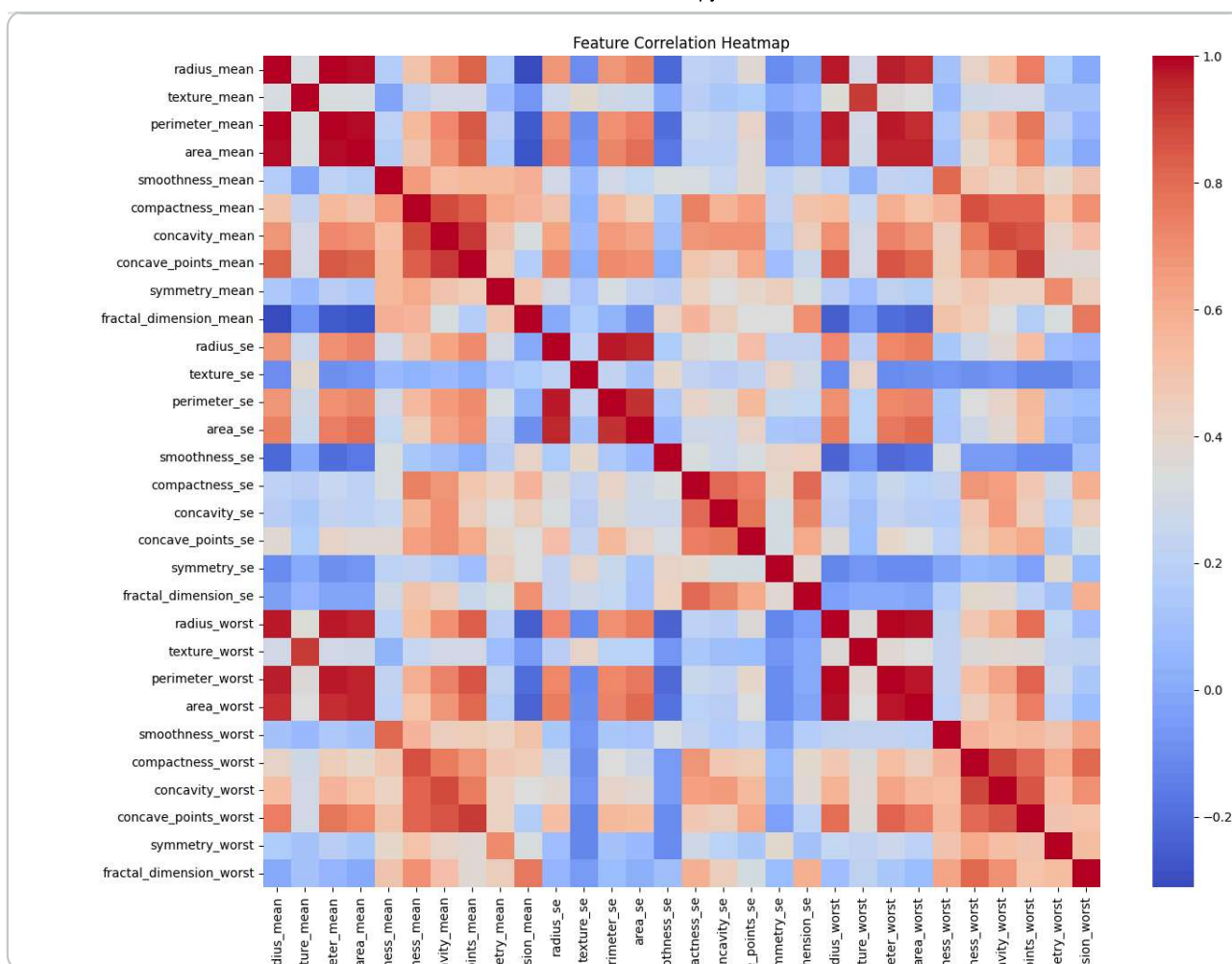


```
plt.figure(figsize=(15, 12))

sns.heatmap(X.corr(), cmap="coolwarm")

plt.title("Feature Correlation Heatmap")

plt.show()
```



```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

```
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
Training samples: 455
Testing samples: 114
```

```
rf = RandomForestClassifier(random_state=42)
```

```
dt = DecisionTreeClassifier(random_state=42)
```

```
dt.fit(X_train, y_train)
```

```
dt_pred = dt.predict(X_test)
```

```
print("Decision Tree Accuracy:",  
      accuracy_score(y_test, dt_pred))
```

Decision Tree Accuracy: 0.9298245614035088

```
dt_params = {  
    "criterion": ["gini", "entropy"],  
    "max_depth": [3, 5, 7, None],  
    "min_samples_split": [2, 5, 10],  
    "min_samples_leaf": [1, 2, 4]  
}
```

```
cv = StratifiedKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=42  
)
```

```
dt_grid = GridSearchCV(  
    DecisionTreeClassifier(random_state=42),  
    dt_params,  
    cv=cv,  
    scoring={  
        "accuracy": "accuracy",  
        "f1": "f1"  
    },  
    refit="accuracy",  
    n_jobs=-1  
)  
  
dt_grid.fit(X_train, y_train)  
  
print("Best Decision Tree Parameters:")  
print(dt_grid.best_params_)  
  
print("\nBest CV Accuracy:",  
      round(dt_grid.cv_results_["mean_test_accuracy"][dt_grid.best_index_] * 100, 2))  
  
print("Best CV F1 Score:",  
      round(dt_grid.cv_results_["mean_test_f1"][dt_grid.best_index_], 4))
```

Best Decision Tree Parameters:
{'criterion': 'entropy', 'max_depth': 5, 'min_samples_leaf': 2, 'min_samples_split': 5}

Best CV Accuracy: 93.63 %
Best CV F1 Score: 0.9121

```

dt_results = pd.DataFrame(dt_grid.cv_results_)

dt_table = pd.DataFrame({
    "Criterion": dt_results["param_criterion"],
    "Max Depth": dt_results["param_max_depth"],
    "Min Samples Split": dt_results["param_min_samples_split"],
    "Min Samples Leaf": dt_results["param_min_samples_leaf"],
    "Avg CV Accuracy (%)": dt_results["mean_test_accuracy"] * 100,
    "Avg CV F1 Score": dt_results["mean_test_f1"]
})

dt_table["Avg CV Accuracy (%)"] = (
    dt_table["Avg CV Accuracy (%)"].round(2)
)

dt_table["Avg CV F1 Score"] = (
    dt_table["Avg CV F1 Score"].round(4)
)

dt_table = dt_table.sort_values(
    "Avg CV Accuracy (%)",
    ascending=False
).reset_index(drop=True)

display(dt_table.head(10))

```



	Criterion	Max Depth	Min Samples Split	Min Samples Leaf	Avg CV Accuracy (%)	Avg CV F1 Score
0	entropy	5	10	2	93.63	0.9121
1	entropy	7	10	2	93.41	0.9094
2	entropy	None	10	2	93.41	0.9094
3	entropy	7	2	2	92.97	0.9042
4	gini	7	5	2	92.97	0.9039
5	entropy	7	5	2	92.97	0.9035
6	gini	None	5	2	92.97	0.9039
7	entropy	5	10	1	92.97	0.9039
8	entropy	None	5	2	92.97	0.9035
9	entropy	None	2	2	92.97	0.9042

```

best_dt = dt_grid.best_estimator_

best_dt.fit(X_train, y_train)

```



```
dt_pred = best_dt.predict(X_test)

print("Best Decision Tree Parameters:")
print(dt_grid.best_params_)

print("\nTest Accuracy:",
      round(accuracy_score(y_test, dt_pred) * 100, 2), "%")
```

```
Best Decision Tree Parameters:
{'criterion': 'entropy', 'max_depth': 5, 'min_samples_leaf': 2, 'min_samples_split': 2}

Test Accuracy: 96.49 %
```

```
rf = RandomForestClassifier(random_state=42)
```

```
rf_params = {
    "n_estimators": [50, 100, 200],
    "max_depth": [3, 5, None],
    "max_features": ["sqrt", "log2"],
    "bootstrap": [True, False]
}
```

```
rf_grid = GridSearchCV(
    rf,
    rf_params,
    cv=cv,
    scoring={
        "accuracy": "accuracy",
        "f1": "f1"
    },
    refit="accuracy",
    n_jobs=-1
)

rf_grid.fit(X_train, y_train)

print("Best Random Forest Parameters:")
print(rf_grid.best_params_)

print("\nBest CV Accuracy:",
      round(rf_grid.cv_results_["mean_test_accuracy"][rf_grid.best_index_] * 100, 2), "%")

print("Best CV F1 Score:",
      round(rf_grid.cv_results_["mean_test_f1"][rf_grid.best_index_], 4))
```

```
Best Random Forest Parameters:
{'bootstrap': False, 'max_depth': 5, 'max_features': 'sqrt', 'n_estimators': 200}
```

Best CV Accuracy: 97.14 %
Best CV F1 Score: 0.9608

```
rf_results = pd.DataFrame(rf_grid.cv_results_)

rf_table = pd.DataFrame({
    "N Estimators": rf_results["param_n_estimators"],
    "Max Depth": rf_results["param_max_depth"],
    "Max Features": rf_results["param_max_features"],
    "Bootstrap": rf_results["param_bootstrap"],
    "Avg CV Accuracy (%)": rf_results["mean_test_accuracy"] * 100,
    "Avg CV F1 Score": rf_results["mean_test_f1"]
})

rf_table["Avg CV Accuracy (%)"] = (
    rf_table["Avg CV Accuracy (%)"].round(2)
)

rf_table["Avg CV F1 Score"] = (
    rf_table["Avg CV F1 Score"].round(4)
)

rf_table = rf_table.sort_values(
    "Avg CV Accuracy (%)",
    ascending=False
).reset_index(drop=True)

display(rf_table.head(10))
```

	N Estimators	Max Depth	Max Features	Bootstrap	Avg CV Accuracy (%)	Avg CV F1 Score
0	200	5	sqrt	False	97.14	0.9608
1	200	None	sqrt	False	97.14	0.9615
2	50	None	sqrt	False	96.92	0.9589
3	100	5	sqrt	False	96.92	0.9580
4	50	None	log2	False	96.92	0.9584
5	100	5	sqrt	True	96.70	0.9553
6	100	5	log2	False	96.70	0.9550
7	100	None	log2	False	96.70	0.9556
8	50	5	sqrt	False	96.48	0.9520
9	200	None	log2	True	96.48	0.9526

```
best_rf = rf_grid.best_estimator_  
  
best_rf.fit(X_train, y_train)  
  
rf_pred = best_rf.predict(X_test)  
  
print("Best Random Forest Parameters:")  
print(rf_grid.best_params_)  
  
print("\nTest Accuracy:",  
      round(accuracy_score(y_test, rf_pred) * 100, 2), "%")
```

```
Best Random Forest Parameters:  
{'bootstrap': False, 'max_depth': 5, 'max_features': 'sqrt', 'n_estimators': 200  
  
Test Accuracy: 96.49 %
```

```
dt_scores = cross_val_score(  
    best_dt,  
    X_train,  
    y_train,  
    cv=cv,  
    scoring="accuracy"  
)  
  
rf_scores = cross_val_score(  
    best_rf,  
    X_train,  
    y_train,  
    cv=cv,  
    scoring="accuracy"  
)  
  
table3 = pd.DataFrame({  
    "Model": [  
        "Decision Tree",  
        "Random Forest"  
    ],  
    "Fold 1": [  
        dt_scores[0] * 100,  
        rf_scores[0] * 100  
    ],  
    "Fold 2": [  
        dt_scores[1] * 100,  
        rf_scores[1] * 100  
    ],  
    "Fold 3": [  
        dt_scores[2] * 100,  
        rf_scores[2] * 100  
    ],  
})
```

```

    "Fold 4": [
        dt_scores[3] * 100,
        rf_scores[3] * 100
    ],
    "Fold 5": [
        dt_scores[4] * 100,
        rf_scores[4] * 100
    ],
    "Average": [
        dt_scores.mean() * 100,
        rf_scores.mean() * 100
    ]
})

table3.iloc[:, 1:] = table3.iloc[:, 1:].round(2)

display(table3)

```

	Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average	
0	Decision Tree	90.11	95.6	91.21	95.6	95.6	93.63	
1	Random Forest	95.60	100.0	94.51	96.7	98.9	97.14	

Next steps:

[Generate code with table3](#)[New interactive sheet](#)

```

dt_pred = best_dt.predict(X_test)
rf_pred = best_rf.predict(X_test)

dt_prob = best_dt.predict_proba(X_test)[:, 1]
rf_prob = best_rf.predict_proba(X_test)[:, 1]

```

```

dt_accuracy = accuracy_score(y_test, dt_pred)
dt_precision = precision_score(y_test, dt_pred)
dt_recall = recall_score(y_test, dt_pred)
dt_f1 = f1_score(y_test, dt_pred)

rf_accuracy = accuracy_score(y_test, rf_pred)
rf_precision = precision_score(y_test, rf_pred)
rf_recall = recall_score(y_test, rf_pred)
rf_f1 = f1_score(y_test, rf_pred)

print("Decision Tree")
print("Accuracy :", round(dt_accuracy, 4))
print("Precision:", round(dt_precision, 4))
print("Recall   :", round(dt_recall, 4))
print("F1 Score :", round(dt_f1, 4))

print("\nRandom Forest")

```

```
print("Accuracy :", round(rf_accuracy, 4))
print("Precision:", round(rf_precision, 4))
print("Recall   :", round(rf_recall, 4))
print("F1 Score :", round(rf_f1, 4))
```

Decision Tree
 Accuracy : 0.9649
 Precision: 1.0
 Recall : 0.9048
 F1 Score : 0.95

Random Forest
 Accuracy : 0.9649
 Precision: 1.0
 Recall : 0.9048
 F1 Score : 0.95

```
final_table = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ],
    "Decision Tree": [
        dt_accuracy,
        dt_precision,
        dt_recall,
        dt_f1
    ],
    "Random Forest": [
        rf_accuracy,
        rf_precision,
        rf_recall,
        rf_f1
    ]
})

final_table.iloc[:, 1:] = final_table.iloc[:, 1:].round(4)

display(final_table)
```

	Metric	Decision Tree	Random Forest	
0	Accuracy	0.9649	0.9649	
1	Precision	1.0000	1.0000	
2	Recall	0.9048	0.9048	
3	F1 Score	0.9500	0.9500	

Next steps:

[Generate code with final_table](#)[New interactive sheet](#)

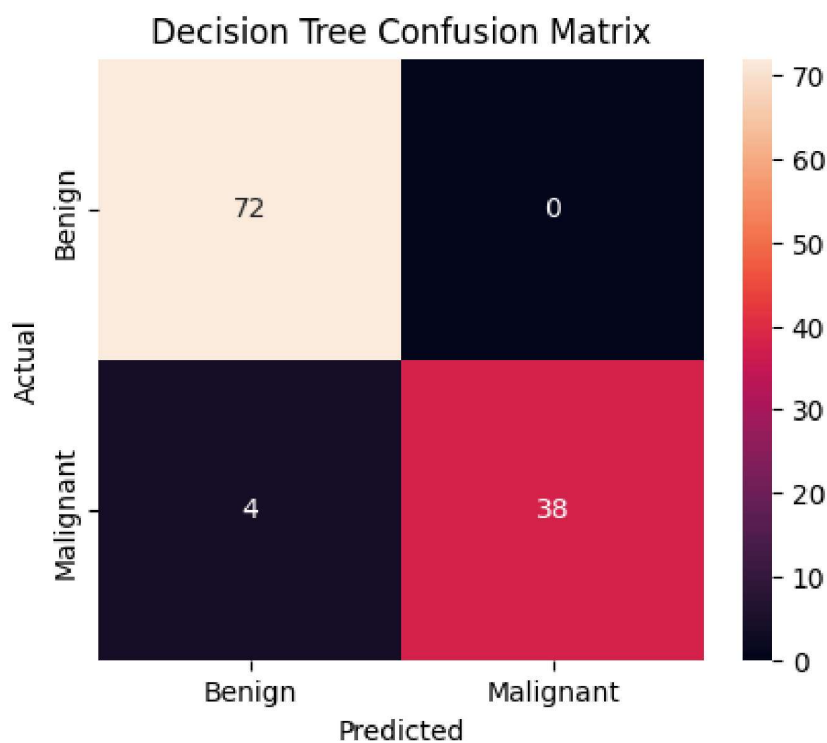
```
cm_dt = confusion_matrix(y_test, dt_pred)

plt.figure(figsize=(5, 4))

sns.heatmap(
    cm_dt,
    annot=True,
    fmt="d",
    xticklabels=["Benign", "Malignant"],
    yticklabels=["Benign", "Malignant"]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Decision Tree Confusion Matrix")

plt.show()
```



```
cm_rf = confusion_matrix(y_test, rf_pred)

plt.figure(figsize=(5, 4))

sns.heatmap(
    cm_rf,
    annot=True,
```

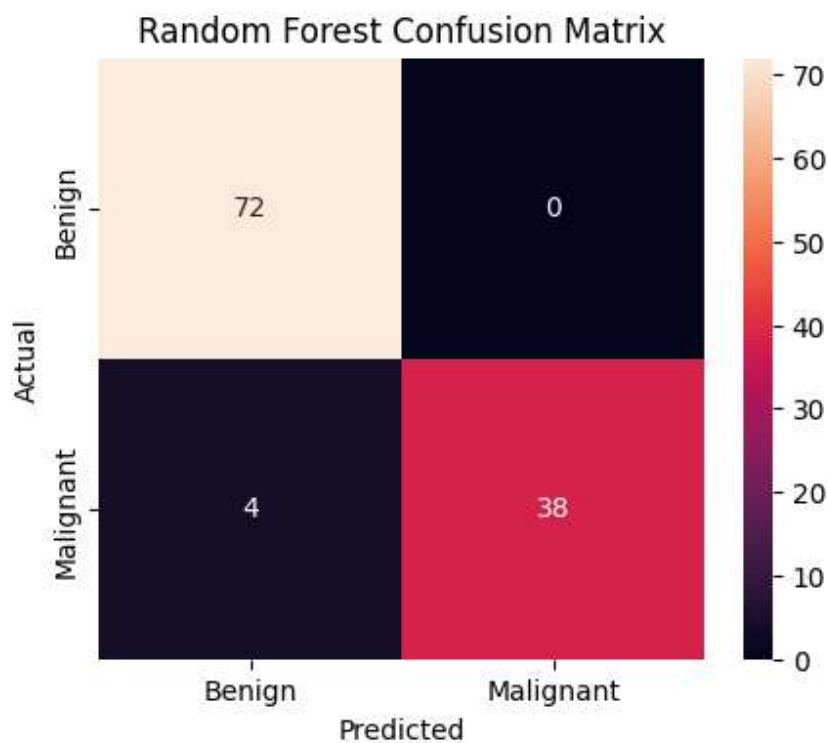
```

    fmt="d",
    xticklabels=["Benign", "Malignant"],
    yticklabels=["Benign", "Malignant"]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Random Forest Confusion Matrix")

plt.show()

```



```

dt_fpr, dt_tpr, _ = roc_curve(y_test, dt_prob)
rf_fpr, rf_tpr, _ = roc_curve(y_test, rf_prob)

dt_auc = roc_auc_score(y_test, dt_prob)
rf_auc = roc_auc_score(y_test, rf_prob)

plt.figure(figsize=(7, 5))

plt.plot(
    dt_fpr,
    dt_tpr,
    label="Decision Tree (AUC = %.3f)" % dt_auc
)

plt.plot(
    rf_fpr,
    rf_tpr,
    label="Random Forest (AUC = %.3f)" % rf_auc
)

```

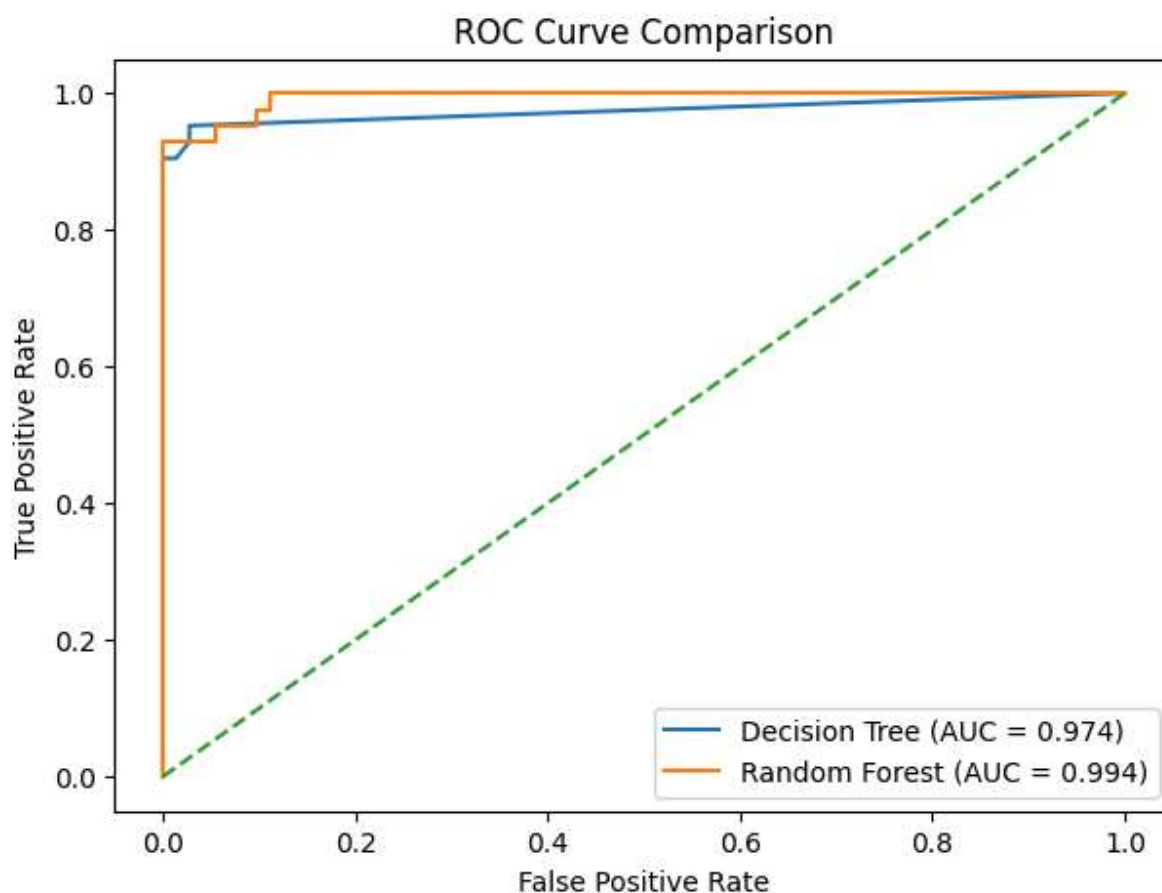
```
)

plt.plot([0, 1], [0, 1], "--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()

plt.show()

print("Decision Tree AUC:", round(dt_auc, 4))
print("Random Forest AUC:", round(rf_auc, 4))
```



```
dt_train = best_dt.score(X_train, y_train)
dt_test = best_dt.score(X_test, y_test)

rf_train = best_rf.score(X_train, y_train)
rf_test = best_rf.score(X_test, y_test)

comparison = pd.DataFrame({
    "Model": [
        "Decision Tree",
        "Random Forest"
    ],
    "Training Accuracy": [
```



```
        dt_train,
        rf_train
    ],
    "Testing Accuracy": [
        dt_test,
        rf_test
    ]
})

comparison.iloc[:, 1:] = comparison.iloc[:, 1:].round(4)

display(comparison)
```

	Model	Training Accuracy	Testing Accuracy	
0	Decision Tree	0.9758	0.9649	
1	Random Forest	0.9912	0.9649	

Next steps: [Generate code with comparison](#) [New interactive sheet](#)

```
auc_table = pd.DataFrame({
    "Model": [
        "Decision Tree",
        "Random Forest"
    ],
    "AUC": [
        dt_auc,
        rf_auc
    ]
})

auc_table["AUC"] = auc_table["AUC"].round(4)

display(auc_table)
```

	Model	AUC	
0	Decision Tree	0.9744	
1	Random Forest	0.9937	

Next steps: [Generate code with auc_table](#) [New interactive sheet](#)

Start coding or [generate](#) with AI.

Visualizations

The following visualizations were generated.

- Benign and malignant class distribution
- Feature correlation heatmap
- Decision Tree confusion matrix
- Random Forest confusion matrix
- ROC curve comparison of Decision Tree and Random Forest

Performance Tables

The models were evaluated using the following metrics.

- Accuracy
- Precision
- Recall
- F1 Score
- Cross-validation accuracy
- ROC-AUC
- Training and testing accuracy

The best Decision Tree model obtained a cross-validation accuracy of 93.63% and a cross-validation F1 score of 0.9121.

The best Random Forest model obtained a cross-validation accuracy of 97.14% and a cross-validation F1 score of 0.9648.

On the test set, both models obtained an accuracy of 96.49%. The precision for both models was 1.0000,

Conclusion

Decision Tree and Random Forest classifiers were used to classify breast cancer cases as benign or malignant. Hyperparameter tuning using GridSearchCV and five-fold cross-validation was performed to identify suitable model parameters. Both models achieved good classification performance, with Random Forest showing better cross-validation performance and ROC-AUC.

Learning Outcomes

- Understood the working of Decision Tree classification.
- Learned how Random Forest combines multiple decision trees.
- Performed hyperparameter tuning using GridSearchCV.
- Understood the use of Stratified K-Fold cross-validation.
- Evaluated classification models using accuracy, precision, recall, F1 score and ROC-AUC.
- Compared the performance of Decision Tree and Random Forest.